CS336 Assignment 1

从零构建 Transformer 语言模型

完整校订中文版 · 逐页英文对照

CS336 Staff，2026 年春，原文版本 26.0.3。每节中文译文之后紧跟对应的英文原页；原页用于核对公式、插图与代码。题号、函数名、代码与数值保留原样。

许可声明（原文仓库 LICENSE）

Copyright 2025 Stanford University

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

原文第 1 页：作业概述

在本作业中，你将从零构建训练标准 Transformer 语言模型（LM）所需的全部组件，并训练若干模型。

需要实现：

1. 字节对编码（BPE）分词器（第 2 节）。
2. Transformer 语言模型（第 3 节）。
3. 交叉熵损失函数和 AdamW 优化器（第 4 节）。
4. 训练循环，包括序列化和加载模型及优化器状态（第 5 节）。

需要运行：在 TinyStories 上训练 BPE 分词器；用训练好的分词器把数据集编码成整数 ID 序列；在 TinyStories 上训练 Transformer LM；用训练好的模型生成样本并计算困惑度；在 OpenWebText 上训练模型，将所得困惑度提交至排行榜。

可使用的组件：每个组件都应从零构建。不得使用 `torch.nn`、`torch.nn.functional` 或 `torch.optim` 中的任何定义，以下例外：`torch.nn.Parameter`、`torch.nn` 中的容器类（如 `Module`、`ModuleList`、`Sequential` 等）以及 `torch.optim.Optimizer` 基类。可以使用 PyTorch 的其他定义。如不确定某个函数或类是否允许使用，可在 Slack 提问；拿不准时，考虑它是否有违本作业“从零实现”的初衷。[1]

AI 工具政策：AI 可以自主完成作业的许多部分，但这会妨碍深入学习。允许用 AI 回答高层概念问题，或查询函数签名、库 API 等底层编程文档；不允许让 AI 实现作业的任何部分。这包括编程代理（如 Cursor Agents、Codex、Claude Code）和 AI 自动补全（如 Cursor Tab、GitHub Copilot）。使用 AI 代理时，应确保它遵循仓库提供的 `AGENTS.md`；使用聊天机器人时，也应在提示中附上这份要求。

[1] `torch.nn` 容器类的完整列表见 `pytorch.org/docs/stable/nn.html#containers`。

原文第 2 页：代码、提交与数据

课程强烈建议完成作业时关闭 IDE 中的 AI 自动补全（如 Cursor Tab、GitHub Copilot）；普通的函数名补全可以使用。往届学生反馈，关闭 AI 自动补全有助于更深入地理解材料。完整政策见讲义中的链接。

代码结构：作业代码和讲义位于 github.com/stanford-cs336/assignment1-basics。请克隆仓库；课程如有更新，会通知你运行 git pull。

1. cs336_basics/*：在这里编写你的代码。此目录没有现成实现，可自行组织。
2. adapters.py：为每项规定功能（例如 scaled dot-product attention）填写适配函数（如 run_scaled_dot_product_attention），使其调用你自己的实现。这里只应写连接代码，不应包含实质算法逻辑。
3. test_*.py：需要通过的测试。它们调用 adapters.py 中的接口；不要修改测试文件。

提交：运行 make_submission.sh 生成提交用 ZIP。如果有不想打包的数据或 checkpoint，需将其加入脚本的排除列表。向 Gradescope 提交 writeup.pdf（排版好的书面题答案）和 code.zip（你的代码）。如要参加排行榜，向 github.com/stanford-cs336/assignment1-basics-leaderboard 提交 PR；具体步骤见该仓库的 README。

数据：本作业使用预处理后的 TinyStories 与 OpenWebText；两者都是单个大型纯文本文件。选课学生按 compute guide 下载；自学者按本仓库 README 的命令下载。

低资源提示（开头）：讲义的蓝色方框会介绍如何在 GPU 资源较少或没有 GPU 时完成部分任务，例如缩小数据集或模型，以及在 Mac 集成 GPU 或 CPU 上训练。这些提示也能帮助有课程机器可用的学生加快迭代。

原文第 3 页：低资源提示与 Unicode

低资源提示：在 Apple Silicon 或 CPU 上做作业 1。课程组的参考实现能在配备 36 GB 内存的 Apple M4 Max 上训练出生成文本较流畅的语言模型：使用 Metal GPU（MPS）不到 5 分钟，使用 CPU 约 30 分钟。即便不熟悉这些术语，只要笔记本电脑较新、实现正确且高效，你也能训练一个生成简单儿童故事的小模型。后文会说明使用 CPU 或 MPS 时应作哪些调整。

2 字节对编码（BPE）分词器

本部分训练并实现一个字节级 BPE 分词器。任意 Unicode 字符串先表示为字节序列，BPE 在该序列上训练；之后分词器把文本字符串编码为用于语言建模的 token（整数序列）。

2.1 Unicode 标准

Unicode 将字符映射到整数码点。Unicode 17.0（2025 年 9 月发布）定义了 172 种文字中的 159,801 个字符。字符 `s` 的码点是 115，通常记作 U+0073（0073 是 115 的十六进制表示）；字符 `牛` 的码点是 29275。Python 中，`ord()` 把单个 Unicode 字符转为码点整数，`chr()` 则把码点转为对应字符。

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

问题 `unicode1`：理解 Unicode（1 分）

(a) `chr(0)` 返回什么 Unicode 字符？提交一句话。

(b) 这个字符的字符串表示（`__repr__()`）与打印出来的表示有何区别？提交一句话。

(c) 这个字符出现在文本中时会发生什么？可在 Python 解释器中运行 `chr(0)`、`print(chr(0))`，以及下一页的两个例子，看看结果是否符合预期。提交一句话。

原文第 4 页：Unicode 编码

unicode1(c) 的后两个例子：

```
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

2.2 Unicode 编码

直接用 Unicode 码点训练分词器不现实：词表会很大（约 15 万项）且稀疏，许多字符极少出现。Unicode 编码可把一个字符转换成字节序列。Unicode 标准规定 UTF-8、UTF-16、UTF-32 三种编码；互联网上超过 98% 的网页使用 UTF-8。

Python 的 encode() 可把 Unicode 字符串编码为 UTF-8；迭代 bytes 对象（如调用 list()）可读取各字节的整数值；decode() 可把 UTF-8 字节串解码回 Unicode 字符串。以下示例必须保留原始输入与输出：

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<class 'bytes'>
>>> # 获取编码后的字节值，范围为 0 到 255。
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129,
161, 227, 129, 175, 33]
>>> # 一个字节不一定对应一个 Unicode 字符。
>>> print(len(test_string))
13
>>> print(len(utf8_encoded))
23
>>> print(utf8_encoded.decode("utf-8"))
hello! こんにちは!
```

UTF-8 将码点序列（有效码点共有 159,801 个，可用 21 位整数表示）转换为 0 – 255 的字节值序列。只有 256 项的字节词表更容易处理。字节级分词也没有词表外 token 的问题，因为任何输入文本都能编码为 0 – 255 的整数序列。

问题 unicode2：Unicode 编码（3 分）

(a) 为什么更倾向于用 UTF-8 编码后的字节、而非 UTF-16 或 UTF-32 字节来训练分词器？比较不同输入字符串在这些编码下的输出可能有所帮助。提交一至两句话。

(b) 下一页给出一个试图将 UTF-8 字节串解码为 Unicode 字符串的错误函数。它错在哪里？给出使它产生错误结果的输入字节串。提交示例并用一句话解释。

原文第 5 页：子词分词

unicode2(b) 中的错误函数和示例：

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])
>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

unicode2(c)：给出一个无法解码为任何 Unicode 字符的双字节序列，并用一句话解释。

2.3 子词分词

字节级分词虽能缓解词级分词器的词表外问题，却会产生很长的输入序列。含 10 个词的句子在词级模型中可能只有 10 个 token，在字符级模型中却可能有 50 个甚至更多（取决于词长）。较长序列使模型每一步计算量增大，也让字节序列建模面对更长的依赖关系。

子词分词位于词级与字节级之间。纯字节词表有 256 项；子词分词器通过增大词表来更好地压缩输入字节序列。例如原始训练文本中经常出现字节串 b'the'，给它一个词表条目，就能把原来的 3 个 token 压缩为 1 个。

如何挑选这些子词？Sennrich 等人提出使用字节对编码（BPE），即一种反复将最高频字节对合并、并赋予新索引的压缩算法。它增添子词 token，以尽可能压缩输入序列；出现足够频繁的词可能最终成为单个子词单元。

用 BPE 构建词表的子词分词器通常称为 BPE 分词器。本作业实现的是字节级 BPE：词表项是单个字节或合并后的字节序列，兼具处理词表外文本和控制序列长度的优点。构建其词表的过程称为“训练”分词器。

2.4 BPE 分词器训练

训练分三步。词表初始化：词表将字节串 token 一一映射到整数 ID。字节有 256 种可能值，初始词表因此包含全部 256 个单字节 token。

预分词（开头）：原则上可以统计全文中相邻字节的频率，并反复合并最高频的一对；但每合并一次就遍历完整语料，计算开销很高。

原文第 6 页：预分词与 BPE 合并

直接跨语料合并字节还可能产生仅标点不同的 token（如 dog! 和 dog.），它们语义接近却会拥有完全不同的 ID。为避免这种情况，先对语料进行较粗粒度的预分词，以便统计字符对出现的次数。比如预分词 text 出现 10 次，那么其相邻字符 t、e 的计数可直接增加 10，而无须再遍历语料。字节级 BPE 的每个预分词以 UTF-8 字节序列表示。

Sennrich 等人的原始 BPE 实现仅按空格拆分（s.split(" ")）。SentencePiece 一类分词器（如 Llama 1/2 的分词器）仍可见这种做法。现代分词器多采用基于正则表达式的预分词，这是 GPT-2 引入的做法。本作业采用下列来自 github.com/openai/tiktoken/pull/234/files 的正则表达式写法：

```
>>> PAT = r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""
>>> # 需要安装 regex 包
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', "'ll", ' pre', '-', 'tokenize']
```

在正式代码中，应使用 re.finditer，避免在建立“预分词 → 计数”映射时把所有预分词都存下来。

计算 BPE 合并：把输入文本预分词，并将每个预分词表示为 UTF-8 字节序列后，就可以训练 BPE。算法反复统计各字节对，选取最高频的一对（如 ("A", "B")），将它每次出现都合并为新 token "AB"，并把新 token 加入词表。因此最终词表大小 = 初始大小（这里是 256）+ 合并次数。出于效率考虑，训练时不统计跨预分词边界的字节对。[2]

频率相同时，确定性地选择字典序较大的字节对。例如以下四对并列最高频，应合并 ("BA", "A")：

```
>>> max([("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A")])
('BA', 'A')
```

特殊 token（开头）：有些字符串（如 <|endoftext|>）用于表示文档边界等元数据。编码文本时，通常希望把它们作为特殊 token 处理；下页续。

[2] 原始 BPE 会加入词尾 token。本作业训练字节级 BPE 时不加入它，因为词表已包含全部字节，包括空格与标点；这些显式表示也会让学到的合并自然反映词边界。

原文第 7 页：特殊 token 与 BPE 示例

特殊 token 不应拆成多个 token，而应始终作为一个固定 ID。例如 <|endoftext|> 必须保持为单个 token，以便语言模型知道何时停止生成。它们需要加入词表并分配固定 ID。Sennrich 等人的算法 1 给出了一种效率不高的 BPE 训练实现，基本遵循前述步骤；先实现并测试它，有助于检查自己的理解。

示例 bpe_example：BPE 训练。考虑以下语料，并把 <|endoftext|> 作为特殊 token：

```
low low low low low
lower lower widest widest widest
newest newest newest newest newest newest
```

词表：初始化为特殊 token <|endoftext|> 加 256 个字节值。

预分词：为突出合并过程，这里仅按空白字符拆分。统计得到 {low: 5, lower: 2, widest: 3, newest: 6}。方便的表示是 dict[tuple[bytes, ...], int]，如 {(l,o,w): 5, …}。Python 中连单个字节也是 bytes 对象；它没有单独的“单字节类型”，就像没有独立的 char 类型一样。

合并：第一轮统计所有相邻字节对，并将它们在各词中的出现频率相加：{lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6}。('e','s') 与 ('s','t') 并列，按字典序取较大的 ('s','t')。合并后，预分词及计数成为 {(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3, (n,e,w,e,st): 6}。

第二轮最常见的是 (e, st)，计数为 9；合并后成为 {(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,est): 3, (n,e,w,est): 6}。继续可得合并序列 ['s t', 'e st', 'o w', 'l ow', 'w est', 'n e', 'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r']。若只进行 6 次合并，词表包含 [<|endoftext|>, [...256 BYTE CHARS], st, est, ow, low, west, ne]。此时 newest 会被编码为 [ne, west]。

原文第 8 页：BPE 训练实验准备

2.5 试验 BPE 分词器训练

接下来在 TinyStories 上训练字节级 BPE。数据下载说明见第 1 节；开始前建议先浏览数据内容。

并行化预分词：预分词往往是主要瓶颈，可用 Python 内置的 `multiprocessing` 并行处理。切分语料时，建议让每个分块的边界落在某个特殊 token 的开头。可以原样使用仓库的 `cs336_basics/pretokenization_example.py` 中寻找边界的起始代码，再把各块分给不同进程。文档边界本来就不能跨越合并，因此这种切分对本作业始终有效；无需考虑没有 `<|endoftext|>` 的极大语料这一边界情况。

预分词之前移除特殊 token：使用 `re.finditer` 的正则表达式预分词前，要先从语料（并行时为当前分块）中拆出所有特殊 token。必须以特殊 token 为分隔符，让它两侧的文本分别预分词，防止跨越边界合并。例如 `[Doc 1]<|endoftext|>[Doc 2]` 应拆成 `[Doc 1]` 与 `[Doc 2]`，分别处理。特殊 token 是训练时的硬边界，但本身不参与合并频率统计。可以使用 `re.split` 和 `"|".join(special_tokens)`，并用 `re.escape` 正确转义特殊 token 中可能出现的 `|`。`test_train_bpe_special_tokens` 会测试这一点。

优化合并：朴素方法每轮都遍历所有字节对重新计数，因而较慢。一次合并只会改变与被合并字节对重叠的那些相邻对的计数。因此，可索引并缓存所有字节对的计数，每轮增量更新，而不重新遍历全文。这样可显著提速；不过 BPE 的合并阶段在 Python 中无法并行化。

低资源提示：性能分析。用 `cProfile`、`py-spy` 等工具找出实现中的瓶颈，集中优化。

低资源提示：缩小规模（开头）。在完整 TinyStories 上训练前，先用较小的“调试数据集”。例如验证集只有约 2.2 万篇文档，训练集约 212 万篇。开发时可缩小数据集或模型，详见下一页。

原文第 9 页：BPE 训练题目

调试配置应足够大，能呈现正式配置中的相同瓶颈，使优化可迁移；同时又不能大到一次运行耗时过长。

问题 train_bpe：BPE 分词器训练（15 分）

编写函数，输入一个文本文件路径，训练字节级 BPE 分词器。至少支持以下参数：

输入	含义
input_path: str	训练文本文件的路径。
vocab_size: int	正整数，规定最终词表大小上限；计算时包括初始字节词表、合并产生的词项和全部特殊 token。
special_tokens: list[str]	要加入词表的字符串列表。训练时把它们当作硬边界，禁止跨越其范围合并；计算合并统计量时不包含它们。

函数应返回：vocab: dict[int, bytes]，即 token ID 到 token 字节串的映射；以及按创建顺序排列的 merges: list[tuple[bytes, bytes]]，每个二元组 (<token1>, <token2>) 表示一次合并。

先实现测试适配接口 adapters.run_train_bpe，再运行 uv run pytest tests/test_train_bpe.py。实现应通过所有测试。可选项：用 C++（如 cppyy、nanobind）或 Rust（PyO3）实现训练的关键部分，但这可能投入大量时间。这样做时要注意哪些操作会复制 Python 内存、哪些能直接读取，并提供构建说明，或保证仅靠 pyproject.toml 即可构建。GPT-2 的正则表达式在多数其他正则引擎中支持不好，往往很慢；课程组验证 Oniguruma 速度尚可并支持负向前瞻，而 Python 的 regex 包甚至更快。

问题 train_bpe_tinystories：在 TinyStories 上训练 BPE（2 分）

(a) 在 TinyStories 上训练字节级 BPE，词表上限为 10,000，并将 <|endoftext|> 加为特殊 token。将结果词表与合并序列序列化到磁盘。报告训练耗时与内存消耗；词表中最长的 token 是什么？它是否合理？资源要求：无 GPU 时不超过 30 分钟、内存不超过 30 GB。

原文第 10 页：训练实验与编码

接续 train_bpe_tinystories(a)：提示——若预分词使用多进程，并利用以下两点，BPE 训练应能控制在 2 分钟内：（1）数据文件中的 <|endoftext|> 用于分隔文档；（2）它在执行 BPE 合并之前已作为特殊情况处理。提交一至两句话。

(b) 对代码进行性能分析。分词器训练中哪一部分最耗时？提交一至两句话。

接下来在 OpenWebText 上训练字节级 BPE；同样建议先浏览数据内容。

问题 train_bpe_expts_owt：在 OpenWebText 上训练 BPE（2 分）

(a) 词表上限 32,000，在 OpenWebText 上训练字节级 BPE。将词表与合并序列保存到磁盘。词表中最长的 token 是什么？它是否合理？资源要求：无 GPU 时不超过 12 小时、内存不超过 100 GB。提交一至两句话。

(b) 比较 TinyStories 与 OpenWebText 上训练所得分词器的异同。提交一至两句话。

2.6 BPE 分词器：编码与解码

前文实现了由输入文本训练出词表和合并序列的函数；现在要实现能加载它们并在文本与 token ID 之间双向转换的分词器。

2.6.1 编码文本

BPE 编码与训练词表的过程相对应。第一步是预分词：像训练时一样，把输入拆成预分词并分别表示为 UTF-8 字节序列。仅在同一预分词内部合并字节，不跨边界。第二步是应用合并：把训练时得到的合并操作按创建顺序应用到每个预分词。

示例 bpe_encoding（起始）：输入字符串为 'the cat ate'，词表是 {0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'}；合并序列从 [(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a', ⋯] 开始，下页续。

原文第 11 页：BPE 编码、解码与接口

示例的合并序列最后一项为 `(b' a', b't')`。预分词器首先把 `'the cat ate'` 拆成 `['the', ' cat', ' ate']`，然后对每项分别应用合并。

`'the'` 起初是 `[b't', b'h', b'e']`。按训练时的合并顺序，先应用 `(b't', b'h')` 得到 `[b'th', b'e']`，再应用 `(b'th', b'e')` 得到 `[b'the']`；没有可继续应用的合并，故对应 ID 序列为 `[9]`。同样，`' cat'` 合并后是 `[b' c', b'a', b't']`，对应 `[7, 1, 5]`；`' ate'` 合并后是 `[b' at', b'e']`，对应 `[10, 3]`。最终编码结果是 `[9, 7, 1, 5, 10, 3]`。

特殊 token：分词器编码时应正确处理构造时提供的用户自定义特殊 token。

内存：大型文本文件可能无法一次装入内存。应将文件或数据流分成可管理的块，依次处理，使内存复杂度相对于文本大小保持常数，而不是线性增长。必须保证 token 不跨块边界，否则结果会不同于一次性将全文装入内存的朴素方法。

2.6.2 解码文本

把整数 token ID 序列解码回原始文本时，查词表取出各 ID 对应的字节序列，拼接后按 Unicode 解码。任意用户输入的 ID 序列不保证产生有效 Unicode；若有非法字节，应以 Unicode 官方替换字符 `U+FFFD` 代替。`bytes.decode` 的 `errors='replace'` 可自动完成这一处理。[3]

问题 `tokenizer`：实现分词器（15 分）实现 `Tokenizer` 类，给定词表及合并序列，将文本编码为整数 ID，并将 ID 解码为文本；支持用户提供的特殊 token（尚未在词表中时需加入）。推荐接口见下一页。

[3] 替换字符说明见 `en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character`。

原文第 12 页：分词器接口与实验

推荐接口如下；类型签名及名称保持原样：

方法	要求
`__init__(self, vocab, merges, special_tokens=None)`	根据词表、合并序列及可选特殊 token 列表构造分词器。参数类型分别为 `dict[int, bytes]`、`list[tuple[bytes, bytes]]`、`list[str] | None`。
`from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)`	类方法，从训练代码保存的词表与合并文件中构造并返回分词器；文件路径均为 `str`，特殊 token 参数为 `list[str] | None`。
`encode(self, text: str) -> list[int]`	将文本编码为 token ID 序列。
`encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]`	给定可迭代的字符串（例如 Python 文件句柄），惰性地产生 ID；大文件内存高效分词需要这一接口。
`decode(self, ids: list[int]) -> str`	将 ID 序列解码为文本。

先实现 `adapters.get_tokenizer`，然后运行 `uv run pytest tests/test_tokenizer.py`；实现应通过全部测试。

2.7 实验

问题 `tokenizer_experiments`：分词器实验（4 分）

(a) 分别从 TinyStories 和 OpenWebText 各抽取 10 篇文档，用之前训练的相应分词器（词表大小分别为 10K、32K）编码成整数 ID。各分词器的压缩比（字节/token）是多少？提交一至两句话。

(b) 用 TinyStories 分词器处理 OpenWebText 样本会怎样？比较压缩比，或定性描述现象。提交一至两句话。

(c) 估计分词器的吞吐量（如字节/秒），推算处理 825 GB 文本的 Pile 数据集需要多久。提交一至两句话。

(d) 用各自分词器将 TinyStories 与 OpenWebText 的训练集和开发集编码成整数 ID 序列，供后续训练语言模型使用。推荐把 ID 序列保存为 `uint16` 类型的 NumPy 数组。为什么 `uint16` 合适？提交一至两句话。

原文第 13 页：Transformer 语言模型

3 Transformer 语言模型架构

语言模型输入一批整数 token ID，张量形状为 (batch_size, sequence_length)；输出词表上的归一化概率分布，形状为 (batch_size, sequence_length, vocab_size)。每个输入位置的分布预测下一个 token。训练时，将预测与实际下一个 token 计算交叉熵损失；推理生成时，取最后一个时间步的预测分布，选取最高概率 token 或按分布采样，接到输入末尾，再重复。

本部分将从零构建该 Transformer LM，先介绍整体结构，再逐步说明各组件。

3.1 Transformer LM

给定 token ID 序列，模型先用输入嵌入将 ID 转成稠密向量，再通过 num_layers 个 Transformer block，最后通过学习得到的线性投影（“输出嵌入”或 “LM head”）产生下一个 token 的 logits。原文图 1 展示整体流程，图 2 展示一个 pre-norm Transformer block；图形及箭头请见对照版英文原页。

原文第 14 页：嵌入、Block 与批量计算

Token 嵌入：首先把一批 token ID 转为包含 token 身份信息的向量序列（原文图 1 的红色方块）。嵌入层输入 (batch_size, sequence_length) 的整数张量，输出 (batch_size, sequence_length, d_model) 的向量序列。

Pre-norm Transformer block：嵌入后的激活值经过多个结构相同的神经网络层。标准 decoder-only Transformer LM 包含 num_layers 个这样的 block。每个 block 输入、输出形状都是 (batch_size, sequence_length, d_model)；它用 self-attention 聚合序列信息，再用前馈层进行非线性变换。经过所有 block 后，将最终激活值转换为词表上的分布。本作业采用 pre-norm block（详见 3.4 节），因此最后一个 block 之后还要做层归一化，以适当缩放输出。随后用学习得到的标准线性变换产生下一个 token 的 logits（参见 Radford 等人文献 [7] 的公式 2）。

3.2 批量计算、einsum 与效率说明

Transformer 中，相同计算会作用于许多“类似批次”的维度：批次中的每个样本都经过相同的前向计算；RMSNorm 和前馈网络等逐位置操作作用于序列每个位置；多头注意力则沿注意力头批量执行。我们希望写法易读、便于理解，又能充分利用 GPU。许多 PyTorch 操作允许张量开头有额外批量维度，并高效地在这些维度上重复或广播计算。

例如数据张量 D 的形状是 (batch_size, sequence_length, d_model)，矩阵 A 是 (d_model, d_model)；D @ A 会沿前两个维度执行高效的批量矩阵乘法。因此，编写函数时应考虑输入可能多出类似批次的前置维度，并把它们留在形状开头。为此，可能需要多次 view、reshape、transpose；这样写有时繁琐，张量形状也不易读。

原文第 15 页：einsum 写法与示例

更方便的方式是 torch.einsum 的 einsum 记法，或使用不绑定框架的 einops、einx。einsum 可在任意输入维度上做张量缩并；rearrange 可重排、连接及拆分维度。机器学习中的许多操作可看作维度重排、张量缩并和少量（通常逐元素的）非线性函数的组合，einsum 记法因而能提高可读性与灵活性。

课程强烈建议学习并使用 einsum 记法。未接触过的同学可先读 einops 文档；熟悉 einops 的同学可进一步学习更通用的 einx。两者已包含在课程提供的环境中。以下示例是 einops 文档的补充，应先读文档。

示例 einstein_example1：用 einops.einsum 做批量矩阵乘法。代码中的字符串和变量名不得翻译：

```
import torch
from einops import rearrange, einsum

## Basic implementation
Y = D @ A.T
# Hard to tell the input and output shapes and what they mean.
# What shapes can D and A have, and do any of these have unexpected behavior?

## Einsum is self-documenting and robust
#                         D              A     ->          Y
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## Or, a batched version where D can have any leading dimensions but A is constrained.
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

示例 einstein_example2：用 einops.rearrange 做广播。对一批图像中的每幅图像，按不同缩放因子生成 10 个变暗版本：

```
images = torch.randn(64, 128, 128, 3)  # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## Reshape and multiply
dim_value = rearrange(dim_by,     "dim_value              -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel")
dimmed_images = images_rearr * dim_value

## Or in one go:
dimmed_images = einsum(
    images, dim_by,
    "batch height width channel, dim_value -> batch dim_value height width channel"
)
```

[4] einops 支持广泛，而 einx 经实际项目检验较少。如发现 einx 的限制或错误，可改用 einops 和普通 PyTorch 操作。

原文第 16 页：重排与像素混合

示例 `einstein_example3`：用 `einops.rearrange` 混合像素。有一批形状为 `(batch, height, width, channel)` 的图像。希望对图像全部像素做线性变换，但每个通道独立处理；矩阵 `B` 的形状为 `(height * width, height * width)`。

```
channels_last = torch.randn(64, 32, 32, 3)  # (batch, height, width, channel)
B = torch.randn(32*32, 32*32)

## Rearrange an image tensor for mixing across all pixels
channels_last_flat = channels_last.view(
    -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
)
channels_first_flat = channels_last_flat.transpose(1, 2)
channels_first_flat_transformed = channels_first_flat @ B.T
channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)
channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)
```

改用 `einops`：

```
height = width = 32
## Rearrange replaces clunky torch view + transpose
channels_first = rearrange(
    channels_last,
    "batch height width channel -> batch channel (height width)"
)
channels_first_transformed = einsum(
    channels_first, B,
    "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
)
channels_last_transformed = rearrange(
    channels_first_transformed,
    "batch channel (height width) -> batch height width channel",
    height=height, width=width
)
```

也可以用 `einx.dot`（`einx` 中与 `einops.einsum` 相应的操作）一次完成：

```
height = width = 32
channels_last_transformed = einx.dot(
    "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
    "-> batch row_out col_out channel",
    channels_last, B,
    col_in=width, col_out=width
)
```

第一种写法虽可在前后加注释说明形状，仍然繁琐且容易出错；einsum 记法本身就能说明计算。它既支持任意前置批量维度，又让输入输出的相关形状一目了然。

原文第 17 页：数学记法与参数初始化

其他张量可用类型提示表明形状，例如 jaxtyping（并不限于 JAX）。einsum 记法的性能影响会在作业 2 讨论；现阶段它几乎总比冗长的重排写法更合适。

3.2.1 数学记法与内存顺序

许多机器学习论文采用行向量记法，与 NumPy 和 PyTorch 默认的行优先内存布局相配。线性变换写为 $y = x W^T$（公式 1），其中 $W \in \mathbb{R}^{d_{out} \times d_{in}}$，行向量 $x \in \mathbb{R}^{1 \times d_{in}}$。扩展 x 的最外层维度即可批量处理：用矩阵 $X \in \mathbb{R}^{batch \times d_{in}}$ 代替单个向量。

线性代数中更常见列向量记法，变换写为 $y = W x$（公式 2），其中同样的行优先矩阵 $W \in \mathbb{R}^{d_{out} \times d_{in}}$，列向量 $x \in \mathbb{R}^{d_{in}}$。此时批量维度需位于最后，输入变为 $X^{\sim} \in \mathbb{R}^{d_{in} \times batch}$。

本作业的数学公式主要采用列向量记法。因此用 PyTorch 的普通矩阵乘法实现时，要按公式 1 的行向量约定对权重转置；若使用 einsum 并正确标记各轴，就不易混淆。Matlab、Julia、Fortran 等通常使用列优先内存布局，批量维度在最后；Python 及相关库遵循 C 的行优先布局。

3.3 基础组件：Linear 与 Embedding

3.3.1 参数初始化

不合适的初始化可能导致梯度消失或爆炸。Pre-norm Transformer 对初始化较稳健，但初始化仍影响训练速度与收敛。本作业先使用以下近似方案，细节留待作业 3：

- Linear 权重：均值 $\mu = 0$、方差 $\sigma^2 = 2/(d_{in} + d_{out})$ 的正态分布，截断到 $[-3\sigma, 3\sigma]$。
- Embedding：均值 $\mu = 0$、方差 $\sigma^2 = 1$ 的正态分布，截断到 $[-3, 3]$。
- RMSNorm：初始化为全 1。

截断正态初始化使用 torch.nn.init.trunc_normal_。

3.3.2 Linear 模块（开头）

线性层是 Transformer 和一般神经网络的基础。需要实现继承 torch.nn.Module 的 Linear 类，完成线性变换；公式见下一页。

原文第 18 页：Linear 与 Embedding 题目

线性变换为 y = W x（公式 3）。与多数现代 LLM 一样，这里不使用偏置项。

问题 linear：实现线性模块（1 分）实现继承 torch.nn.Module 的 Linear 类，接口与 PyTorch 内置 nn.Linear 相近，但没有 bias 参数或偏置项。推荐构造函数 __init__(self, in_features, out_features, device=None, dtype=None)：in_features、out_features 分别是输入、输出最后一维的大小；device 和 dtype 指定参数设备、数据类型。forward(self, x: torch.Tensor) -> torch.Tensor 执行线性变换。

必须继承 nn.Module、调用父类构造函数、把参数存为 W（而非 WT）并置于 nn.Parameter 中；不得使用 nn.Linear 或 nn.functional.linear。按上一页设置并用 torch.nn.init.trunc_normal_ 初始化。测试适配函数 adapters.run_linear 应把给定权重加载进模块（可用 Module.load_state_dict），然后运行 uv run pytest -k test_linear。

3.3.3 Embedding 模块

Transformer 的第一层将整数 token ID 映射到 d_model 维向量空间。自定义 Embedding 类须继承 torch.nn.Module，不得使用 nn.Embedding。前向方法应使用形状 (batch_size, sequence_length) 的 torch.LongTensor ID，从形状 (vocab_size, d_model) 的嵌入矩阵中索引各 token 对应的向量。

问题 embedding：实现嵌入模块（1 分）实现继承 torch.nn.Module 的 Embedding，执行嵌入查找，接口尽量与内置 nn.Embedding 一致；具体参数要求见下一页。

原文第 19 页：Embedding、Pre-norm 与 RMSNorm

Embedding 的推荐接口：__init__(self, num_embeddings, embedding_dim, device=None, dtype=None)。num_embeddings 是词表大小，embedding_dim 是嵌入维度 d_model；device、dtype 指定参数设备及数据类型。forward(self, token_ids: torch.Tensor) -> torch.Tensor 查找给定 ID 的嵌入向量。

须继承 nn.Module、调用父类构造函数、将嵌入矩阵初始化为 nn.Parameter，且让 d_model 位于矩阵最后一维；不得使用 nn.Embedding 或 nn.functional.embedding。沿用前述初始化参数及 torch.nn.init.trunc_normal_。实现 adapters.run_embedding 后运行 uv run pytest -k test_embedding。

3.4 Pre-norm Transformer block

每个 block 有两个子层：多头自注意力与逐位置前馈网络。原始 Transformer 在各子层外加残差连接，然后做层归一化，称为 post-norm。研究发现，将层归一化移到每个子层的输入端，并在最后一个 block 后再加一次归一化，可改善训练稳定性；这称为 pre-norm（见原文图 2）。每个子层输出都通过残差连接与该子层输入相加。直观上，输入嵌入到最终输出之间存在一条未经归一化打断的“残差流”，有利于梯度传播。GPT-3、LLaMA、PaLM 等采用这一形式；本作业也实现它，并逐个介绍其组件。

3.4.1 均方根层归一化

原始 Transformer 使用 LayerNorm；本作业按 Touvron 等人的做法，使用均方根层归一化 RMSNorm。给定激活向量 a ∈ ℝ^(d_model)，第 i 个分量变为 RMSNorm(a)_i = (a_i / RMS(a)) · g_i（公式 4），其中 g_i 是可学习的增益参数。具体的 RMS(a) 定义见下一页。

原文第 20 页：RMSNorm 题目

RMS(a) = sqrt[(1/d_model) · Σ_{i=1}^{d_model} a_i² + ε]。共有 d_model 个可学习的 g_i；超参数 ε 通常固定为 1e-5。对输入求平方前应先转为 torch.float32，避免溢出；归一化完成后转换回输入的原数据类型。讲义给出的前向流程框架如下，省略号仍是待实现部分：

```
in_dtype = x.dtype
x = x.to(torch.float32)
# Your code here performing RMSNorm
...
result = ...
# Return the result in the original dtype
return result.to(in_dtype)
```

问题 rmsnorm：均方根层归一化（1 分）将 RMSNorm 实现为 torch.nn.Module。推荐构造函数 __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)，参数分别是隐藏维度、用于数值稳定的 ε、参数设备与数据类型。forward(self, x: torch.Tensor) -> torch.Tensor 处理形状 (batch_size, sequence_length, d_model) 的输入，返回同形状张量。记得在归一化前上转为 float32，之后下转为原数据类型。实现 adapters.run_rmsnorm 后运行 uv run pytest -k test_rmsnorm。

原文第 21 页：逐位置前馈网络

3.4.2 逐位置前馈网络

图 3 比较 SiLU（也称 Swish）与 ReLU 激活函数。原始 Transformer 的前馈网络由两次线性变换构成，中间使用 $\mathrm{ReLU}(x) = \max(0, x)$；内部层维度通常为输入维度的 4 倍。

现代语言模型通常改用另一种激活函数，并引入门控机制。本作业实现 Llama 3、Qwen 2.5 等模型所用的 SwiGLU：它结合 SiLU/Swish 激活与门控线性单元（GLU）。依照 PaLM、LLaMA 以来的常见做法，线性层不使用偏置。

SiLU 的定义（公式 5）为 $\mathrm{SiLU}(x) = x \cdot \mathrm{sigmoid}(x) = x / (1 + e^{-x})$。它与 ReLU 相似，但在零点是平滑的。

GLU 最初定义为两条线性分支的逐元素乘积，其中一条经过 sigmoid（公式 6）：$\mathrm{GLU}(x, W_1, W_2) = \mathrm{sigmoid}(W_1 x) \odot W_2 x$。$\odot$ 表示逐元素相乘。门控分支保留非线性表达能力，另一路为梯度提供线性通道，有助于缓解深层网络的梯度消失。

原文第 22 页：SwiGLU 与旋转位置编码

组合 SiLU 与 GLU，就得到本作业前馈网络（公式 7）：$\mathrm{FFN}(x) = \mathrm{SwiGLU}(x, W_1, W_2, W_3) = W_2[\mathrm{SiLU}(W_1 x) \odot W_3 x]$。其中 $x \in \mathbb{R}^{d_model}$，$W_1, W_3 \in \mathbb{R}^{d_ff \times d_model}$，$W_2 \in \mathbb{R}^{d_model \times d_ff}$。常用 $d_ff = (8/3) d_model$；实现时可以取附近的 64 的倍数，以提高硬件效率。

Shazeer 首先提出将 SiLU/Swish 与 GLU 结合，并在语言建模实验中观察到 SwiGLU 优于 ReLU 和无门控的 SiLU。作业后面会要求比较 SwiGLU 与 SiLU。作者也提醒读者保持经验性的视角；论文中幽默地说：“我们无法解释这些架构为何似乎有效；同其他一切一样，我们将它们的成功归功于神明的仁慈。”

问题 positionwise_feedforward：实现逐位置前馈网络（2 分）实现由 SiLU 激活和 GLU 组成的 SwiGLU 前馈网络。可使用 torch.sigmoid 以获得数值稳定性。将 d_ff 设为约 $(8/3) d_model$，且内部层维度应为 64 的倍数。完成 adapters.run_swiglu，运行 uv run pytest -k test_swiglu。

3.4.3 相对位置嵌入

为模型注入位置信息，本作业实现旋转位置嵌入 RoPE。对位置 i 的查询向量 $q^{(i)} = W_q x^{(i)} \in \mathbb{R}^d$，施加旋转矩阵 R_i，得到 $q'^{(i)} = R_i q^{(i)} = R_i W_q x^{(i)}$。它把向量中相邻的两个分量 $q^{(i)}_{(2k-1:2k)}$ 视作二维向量，以角度 $\theta_{(i,k)} = i / \Theta^{(2k-2)/d}$ 旋转，其中 $k \in \{1, \dots, d/2\}$，Θ 是常数。整个 $d \times d$ 矩阵 R_i 是分块对角矩阵，每个 2×2 块（公式 8）为：

```
R_(i,k) = [[cos θ_(i,k), -sin θ_(i,k)],
           [sin θ_(i,k),  cos θ_(i,k)]]
```

原文第 23 页：RoPE 与注意力的 Softmax

完整的旋转矩阵 R_i（公式 9）将各 R_(i,k) 放在对角线上，非对角块为 2×2 零矩阵。实现时不必显式构造整个 d×d 矩阵，应利用其结构高效地进行成对旋转。不同层、不同 batch 可复用计算好的正弦与余弦值。可在初始化时用 self.register_buffer(persistent=False) 保存二维预计算表，再让所有层引用同一个 RoPE 模块；这些固定值并非可学习参数，因此不应放在 nn.Parameter 中。对位置 j 的键向量 $k^{(j)}$，同样施加其对应的 R_j。RoPE 本身没有可学习参数。

问题 rope：实现 RoPE（2 分）实现 RotaryPositionalEmbedding 类。推荐接口：__init__(self, theta: float, d_k: int, max_seq_len: int, device=None)，分别传入 Θ、查询/键向量维度、最大序列长度、缓冲区设备；forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor 接收形状 (..., seq_len, d_k) 的张量并返回同形状结果。输入可以有任意数量的 batch 维；token_positions 形状是 (..., seq_len)，指定序列维各元素的位置。应按这些位置索引预计算的正弦/余弦张量。完成 adapters.run_rope 并运行 uv run pytest -k test_rope。

3.4.4 缩放点积注意力

首先定义 softmax：将未经归一化的分数向量变为概率分布。公式 10 为 softmax(v)_i = exp(v_i) / Σ_(j=1)^n exp(v_j)。

原文第 24 页：Softmax、掩码与缩放点积注意力

当 v_i 很大时，exp(v_i) 可能溢出为无穷大，造成 ∞/∞ = NaN。因为所有输入同时加同一个常数不会改变 softmax，可在计算前减去该维的最大值，让最大的新值为零，以提高数值稳定性。

问题 softmax：实现 Softmax（1 分）函数接收张量和维度 i，沿第 i 维应用 softmax；返回同形状张量，其中第 i 维成为归一化概率分布。使用减去该维最大值的技巧。完成 adapters.run_softmax，运行 uv run pytest -k test_softmax_matches_pytorch。

注意力运算（公式 11）：Attention(Q, K, V) = softmax(QKT / √d_k) V，其中 Q ∈ ℝ^(n×d_k)、K ∈ ℝ^(m×d_k)、V ∈ ℝ^(m×d_v)。这里的 Q, K, V 是运算的输入，不是可学习参数。

掩码：布尔矩阵 M ∈ {True, False}^(n×m) 的第 i 行指示第 i 个查询可关注哪些键。M_(i,j)=True 表示该查询可以关注第 j 个键，False 表示不可关注。例如 [[True, True, False]] 使唯一的查询只关注前两个键。计算时，在 softmax 之前把掩码为 False 的位置加上 −∞，无需为每个子序列另算一次注意力。

问题 scaled_dot_product_attention：实现缩放点积注意力（5 分）键、查询形状为 (batch_size, ..., seq_len, d_k)，值形状为 (batch_size, ..., seq_len, d_v)；... 表示任意数量的额外 batch 类维。输出形状 (batch_size, ..., seq_len, d_v)。还应支持用户可选的布尔掩码 (seq_len, seq_len)：掩码为 True 的位置的注意力概率加总为 1，False 的位置概率为 0。

原文第 25 页：多头自注意力与因果掩码

完成 adapters.run_scaled_dot_product_attention。分别运行 uv run pytest -k test_scaled_dot_product_attention（三阶输入）和 uv run pytest -k test_4d_scaled_dot_product_attention（四阶输入）。

3.4.5 因果多头自注意力

多头注意力（公式 12、13）为 $\mathrm{MultiHead}(Q,K,V) = \mathrm{Concat}(\mathrm{head}_1,\dots,\mathrm{head}_h)$，其中 $\mathrm{head}_i = \mathrm{Attention}(Q_i,K_i,V_i)$。每个 Q_i,K_i,V_i 是相应向量的第 i 个头切片，大小为 d_k 或 d_v。因此，多头自注意力（公式 14）是 $\mathrm{MultiHeadSelfAttention}(x) = W_O\,\mathrm{MultiHead}(W_Q x, W_K x, W_V x)$。可学习参数形状：$W_Q, W_K \in \mathbb{R}^{(h\,d_k \times d_{model})}$，$W_V \in \mathbb{R}^{(h\,d_v \times d_{model})}$，$W_O \in \mathbb{R}^{(d_{model} \times h\,d_v)}$。$W_Q, W_K, W_V$ 的输出维可按头拆开。正确实现后，查询、键、值投影总共应只需三次矩阵乘法。扩展挑战：把三种投影合并成一个权重矩阵，从而只用一次矩阵乘法。

因果掩码：预测前缀 $t_1,\dots,t_i$ 后的下一个 token 时，模型不能看到未来位置 $t_{(i+1)},\dots,t_n$，否则训练时会泄露真实答案。可以使用一次带因果掩码的多头自注意力：位置 i 仅关注 $j \le i$ 的位置。可用 torch.triu 或广播后的索引比较构造掩码，并复用前一节支持掩码的缩放点积注意力。

应用 RoPE：仅旋转查询与键向量，不旋转值向量。头维应作为 batch 类维处理，让每个头的查询和键按相同的位置接受同样的旋转。

问题 multihead_self_attention：实现因果多头自注意力（5 分）具体参数见下一页。

原文第 26 页：Transformer Block 与完整语言模型

将因果多头自注意力实现为 `torch.nn.Module`，至少接收 `d_model: int`（block 输入维度）与 `num_heads: int`（头数）。按原始 Transformer，令 `d_k = d_v = d_model / h`。完成 `adapters.run_multihead_self_attention`，运行 `uv run pytest -k test_multihead_self_attention`。

3.5 完整的 Transformer 语言模型

每个 Transformer block 有两个子层：多头自注意力与 SwiGLU 前馈网络。每个子层依次执行 RMSNorm、主要运算（MHA 或 FFN），最后加上残差。第一个子层（公式 15）是 `y = x + MultiHeadSelfAttention(RMSNorm(x))`。

问题 `transformer_block`：实现 Transformer block（3 分）依第 3.4 节和原文图 2 实现 pre-norm block。至少接收 `d_model: int`（block 输入维度）、`num_heads: int`（头数）、`d_ff: int`（前馈网络内部维度）。完成 `adapters.run_transformer_block` 并运行 `uv run pytest -k test_transformer_block`。交付物是通过所给测试的 block 代码。

按照原文图 1 拼装模型：先按第 3.1.0.1 节的描述进行嵌入，接着经过 `num_layers` 个 Transformer block，再经最后的层归一化和 LM head，得到对整个词表的未归一化分数（logits）。

问题 `transformer_lm`：实现 Transformer 语言模型（3 分）按第 3.1 节及原文图 1 实现。除 block 的构造参数外，至少还须接收 `vocab_size: int`，用以确定 token 嵌入矩阵的维度；其余要求见下一页。

原文第 27 页：模型资源核算

完整模型还须接收 context_length: int（最大上下文长度，用于确定 RoPE 正弦/余弦缓冲区大小）和 num_layers: int（Transformer block 数）。完成 adapters.run_transformer_lm，运行 uv run pytest -k test_transformer_lm。交付物是通过这些测试的 Transformer LM 模块。

资源核算

了解 Transformer 各部分如何使用计算量与内存很有帮助。主要的浮点运算量来自矩阵乘法，因此：（1）列出一次前向传播中的所有矩阵乘法；（2）把各次乘法换算成 FLOPs。规则：若 $A \in \mathbb{R}^{m\times n}$、$B \in \mathbb{R}^{n\times p}$，则 AB 需要 2mnp FLOPs。原因是输出的 mp 个元素各需 n 次乘法和 n 次加法。

问题 transformer_accounting：Transformer LM 资源核算（5 分）

- (a) 使用本作业的架构、GPT-2 XL 规模的配置：vocab_size=50,257，context_length=1,024，num_layers=48，d_model=1,600，num_heads=25，d_ff=4,288（最接近 $(8/3)\times 1{,}600$ 的 64 的倍数）。模型共有多少可训练参数？假设每个参数为单精度浮点数，仅加载模型需要多少内存？交付：1–2 句话。
- (b) 对上下文长度为 context_length 个 token 的输入，完成此前向传播需哪些矩阵乘法？合计多少 FLOPs？交付：附说明的矩阵乘法列表及 FLOPs 总数。后续小题见下一页。

原文第 28 页：资源核算续与训练概述

- (c) 根据上题分析，模型中哪些部分消耗最多 FLOPs？交付：1 – 2 句话。
- (d) 对 GPT-2 small（12 层，d_model=768，12 个头）、medium（24 层，1024，16 个头）、large（36 层，1280，20 个头）重复上述分析。模型变大时，各部分占总 FLOPs 的比例如何变化？交付：每个模型各组件 FLOPs 及其占总量的比例，再用 1 – 2 句话描述变化。
- (e) 将 GPT-2 XL 的上下文长度增至 16,384：一次前向传播的总 FLOPs 如何变化？各组件的相对贡献如何变化？交付：1 – 2 句话。

4 训练 Transformer 语言模型

数据预处理（tokenizer）与模型（Transformer）已有设计，接下来需要训练所需代码：交叉熵损失、AdamW 优化器，以及负责加载数据、保存检查点、管理训练的循环。

4.1 交叉熵损失

对每个长度为 m+1 的序列 x，语言模型给出 p_θ(x_(i+1)|x_(1:i))，i=1,…,m。训练集 D 由这样的序列组成；标准交叉熵（负对数似然）损失（公式 16）为：ℓ(θ;D) = [1/(|D|m)] Σ_(x∈D) Σ_(i=1)^m [-log p_θ(x_(i+1)|x_(1:i))]。Transformer 一次前向传播即可产生所有位置的预测分布。

位置 i 的 logits 为 o_i ∈ ℝ^(vocab_size)，从而（公式 17）p(x_(i+1)|x_(1:i)) = softmax(o_i)[x_(i+1)] = exp(o_i[x_(i+1)]) / Σ_(a=1)^(vocab_size) exp(o_i[a])。其中 o_i[k] 指向量 o_i 第 k 个索引处的值；交叉熵对应真实目标的点质量分布与预测 softmax 分布之间的交叉熵。

原文第 29 页：交叉熵、困惑度与 SGD

实现交叉熵时，也需像 softmax 一样注意数值稳定性。

问题 cross_entropy：实现交叉熵（1 分）函数接收预测 logits o_i 与目标 x_(i+1)，计算 ℓ_i = -log softmax(o_i)[x_(i+1)]。要求：减去最大元素以稳定数值；能消去 log 和 exp 的地方尽量消去；处理任意额外 batch 维并在 batch 范围取平均。batch 类维总在词表大小维之前。完成 adapters.run_cross_entropy，运行 uv run pytest -k test_cross_entropy。

困惑度：训练用交叉熵，评估时还报告 perplexity。若长度 m 的序列对应损失 $\ell_1, \cdots, \ell$_m，公式 18 为 perplexity = exp[(1/m) Σ_(i=1)^m ℓ_i]。

4.2 SGD 优化器

随机梯度下降从随机参数 θ_0 开始，在每步 t=0,⋯,T-1 更新（公式 19）：θ_(t+1) ← θ_t − α_t ∇L(θ_t; B_t)。B_t 是从数据集 D 随机抽取的 batch；学习率 α_t 和 batch 大小 |B_t| 是超参数。

4.2.1 在 PyTorch 中实现 SGD

自定义优化器继承 torch.optim.Optimizer，须实现两个方法。__init__(self, params, ...) 初始化参数集合或参数组，并将参数与默认超参数字典传给基类；每组可有不同的学习率等。step(self) 在反向传播后遍历各参数张量 p，利用存在时的梯度 p.grad 原地修改其 p.data。下页给出一个学习率随步数衰减的例子。

原文第 30 页：SGD 示例代码

示例更新式（公式 20）为 θ_(t+1) = θ_t − [α / √(t+1)] ∇L(θ_t;B_t)。原文代码如下，代码标识符保持英文：

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
        if lr < 0:
            raise ValueError(f"Invalid learning rate: {lr}")
        defaults = {"lr": lr}
        super().__init__(params, defaults)

    def step(self, closure: Optional[Callable] = None):
        loss = None if closure is None else closure()
        for group in self.param_groups:
            lr = group["lr"]  # Get the learning rate.
            for p in group["params"]:
                if p.grad is None:
                    continue

                state = self.state[p]  # Get state associated with p.
                t = state.get("t", 0)  # Get iteration number from the state, or 0.
                grad = p.grad.data  # Get the gradient of loss with respect to p.
                p.data -= lr / math.sqrt(t + 1) * grad  # Update weight tensor in-place.
                state["t"] = t + 1  # Increment iteration number.

        return loss
```

基类会保存参数和默认超参数。若传入的是一组 `torch.nn.Parameter`，基类会建一个参数组。在 `step` 中逐组、逐参数应用公式 20；每个参数的状态记录自己的迭代次数。API 允许传入 `closure` 以重算损失，本作业的优化器不需要，但示例保留该参数以符合接口。最小训练循环示例如下，后两行在下一页：

```
weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)

for t in range(100):
    opt.zero_grad()  # Reset the gradients for all learnable parameters.
    loss = (weights**2).mean() # Compute a scalar loss value.
    print(loss.cpu().item())
```

原文第 31 页：学习率实验与 AdamW

上页训练循环接着执行：

```
    loss.backward() # Run backward pass, which computes gradients.
    opt.step() # Run optimizer step.
```

语言模型训练也采用此结构。可由 `m.parameters()` 得到模型可学习参数，损失则在采样的 batch 上计算。

问题 `learning_rate_tuning`：调节学习率（1 分）把上面的 SGD 示例只运行 10 次迭代，分别将学习率换成 `1e1`、`1e2`、`1e3`。观察每个学习率下损失是更快或更慢下降，还是随训练发散上升。交付：用 1 – 2 句话说明观察到的行为。

4.3 AdamW

现代语言模型通常使用比 SGD 更复杂的优化器，多数与 Adam 相关。本作业实现广泛使用的 AdamW。它通过与梯度更新解耦的权重衰减改进正则化：每次迭代把参数朝零拉近。实现参考 Loshchilov 等人论文的算法 2。

AdamW 为每个参数保存一阶、二阶矩估计，以额外内存换取稳定性与收敛表现。除了学习率 α，还有控制矩估计的 β_1, β_2，以及权重衰减率 λ。常见值是 $(0.9, 0.999)$；LLaMA、GPT-3 等模型常用 $(0.9, 0.95)$。ε（例如 10^{-8}）用于二阶矩极小时的数值稳定性。算法见下一页。

原文第 32 页：AdamW 算法与内存核算

算法 1：AdamW 优化器（这里 t 从 1 开始；g^2 为逐元素平方）：

```
初始化可学习参数 θ；m ← 0；v ← 0
对 t = 1,…,T：
  抽取数据批次 B_t
  g ← ∇_θ ℓ(θ; B_t)
  α_t ← α √(1−β₂^t)/(1−β₁^t)
  θ ← θ − α λ θ                  # 权重衰减
  m ← β₁m + (1−β₁)g              # 一阶矩
  v ← β₂v + (1−β₂)g²             # 二阶矩
  θ ← θ − α_t m/(√v + ε)         # 矩调整后的更新
```

问题 adamw：实现 AdamW（2 分）继承 torch.optim.Optimizer，构造函数接收学习率 α，以及 β、ε、λ 等超参数。基类的 self.state 把 nn.Parameter 映射到可保存状态的字典，可用于保存矩估计。完成 adapters.get_adamw_cls，运行 uv run pytest -k test_adamw。

问题 adamw_accounting：AdamW 训练资源核算（2 分）假设所有张量为 float32。(a) 运行 AdamW 的峰值内存是多少？按参数、激活、梯度和优化器状态分别核算，并以 batch_size 及模型超参数 vocab_size, context_length, num_layers, d_model, num_heads 表达；假设 d_ff=(8/3)d_model。为了简化激活的内存计算，只考虑：Transformer block 中的 RMSNorm；多头自注意力的 QKV 投影、QKT 矩阵乘法、softmax、值的加权求和、输出投影；逐位置前馈（SwiGLU）的 W_1、W_2、门控分支的 SiLU、逐元素乘积、W_3。后续条件见下一页。

原文第 33 页：内存核算与学习率调度

adamw_accounting 的 (a) 还要计入最终 RMSNorm、输出嵌入层，以及 logits 上的交叉熵。交付：分别给出参数、激活、梯度、优化器状态和总量的代数表达式。

- (b) 将 (a) 代入 GPT-2 XL 规模配置，得到仅依赖 batch_size 的表达式。80 GB 内存最多容纳多大的 batch？交付：数值形式 a · batch_size+b 及最大 batch 大小。
- (c) AdamW 单步更新需要多少 FLOPs？交付：代数表达式与简短依据。
- (d) 模型 FLOPs 利用率 MFU 定义为观察到的吞吐率（token/s）相对于硬件理论峰值 FLOPs 吞吐率的比值。NVIDIA H100 对 “float32”（实际是 TensorFloat-32，原文称其实为 “bfloat19”）运算的理论峰值为 495 teraFLOP/s。假设 MFU 达到 50%，在单张 H100 上以 batch 大小 1024 训练 GPT-2 XL 共 400K 步要多少小时？按 Kaplan、Hoffmann 等人的假设，反向传播 FLOPs 是前向传播的两倍。交付：小时数与简短依据。

4.4 学习率调度

使损失下降最快的学习率可能随训练阶段变化。训练 Transformer 时常先用较大的学习率，之后缓慢衰减。本作业实现 LLaMA 所用的余弦退火调度；也有让学习率重新上升（restart）以帮助越过局部极小值的做法。

调度器是一个函数：输入当前步数 t 和初始/最终学习率等参数，返回当前步更新应使用的学习率。这里输入 t、最大学习率 α_max、最小（最终）学习率 α_min、预热迭代数 T_w、余弦退火结束步 T_c，分段定义为：

```
预热：       t < T_w       时，α_t = (t/T_w) α_max
余弦退火：   T_w ≤ t ≤ T_c 时，α_t = α_min + 0.5[1+cos(π(t-T_w)/(T_c-T_w))](α_max-α_min)
退火结束后： t > T_c       时，α_t = α_min
```

问题 learning_rate_schedule：实现带预热的余弦学习率调度（1 分）具体测试要求见下一页。

原文第 34 页：梯度裁剪与数据加载

编写函数，按上页定义接收 t, α_{max}, α_{min}, T_w, T_c 并返回 α_t。完成 adapters.get_lr_cosine_schedule，运行 uv run pytest -k test_get_lr_cosine_schedule。

4.5 梯度裁剪

某些训练样本可能产生很大的梯度，使训练不稳定。每次反向传播后、优化器更新前，可限制所有参数合并后的梯度向量 g 的 ℓ_2 范数。若 $\|g\|_2 < M$，不变；否则把 g 乘以 $M/(\|g\|_2+\varepsilon)$，其中 ε（例如 10^{-6}）用于数值稳定。缩放后的范数会略低于 M。

问题 gradient_clipping：实现梯度裁剪（1 分）函数接收参数列表和最大 ℓ_2 范数，原地修改每个参数的梯度。使用 $\varepsilon=10^{-6}$（PyTorch 默认值）。完成 adapters.run_gradient_clipping，运行 uv run pytest -k test_gradient_clipping。

5 训练循环

现在把 token 化的数据、模型和优化器组合起来。

5.1 数据加载器

token 化后的数据是一条整数 token 序列 $x=(x_1,\cdots,x_n)$。即使原数据来自多个文档，也常在文档之间加入 <|endoftext|> 之类的分隔符，再拼成一条序列。数据加载器抽取 batch：每批有 B 条长度为 m 的输入序列，以及对应的长度为 m 的下一 token 目标。例如 B=1, m=3 时，$([x_2,x_3,x_4],[x_3,x_4,x_5])$ 是一个样本。

任何 $1\leqslant i\leqslant n-m$ 都可作起点，因此采样简单；序列等长，无需填充，可更充分利用硬件；采样时也无需把整份数据装入内存。

问题 data_loading：实现数据加载（2 分）函数输入 token ID 的整数 NumPy 数组 x、batch_size、context_length 和 PyTorch 设备字符串（例如 'cpu'、'cuda:0'），返回采样输入与对应的下一 token 目标两个张量；形状、设备及测试要求见下一页。

原文第 35 页：内存映射与检查点

data_loading 返回的两个张量都应为 (batch_size, context_length)，包含 token ID，且均位于指定设备上。完成 adapters.run_get_batch，运行 uv run pytest -k test_get_batch。

低资源提示：在 CPU 上训练使用 'cpu'，Apple Silicon 使用 'mps'；数据和模型都需移到对应设备。参考 PyTorch 的 MPS 文档及 Apple Metal Performance Shaders 文档。

若数据集大到无法装入内存，可用 Unix 的 mmap 把磁盘文件映射到虚拟内存，按需读取。NumPy 提供 np.memmap；若用 np.save 保存，可用 np.load(..., mmap_mode='r')。训练采样时应以这种内存映射模式加载数据，并指定与实际数组匹配的 dtype。可检查读出的 token ID 是否在预期词表范围内。

5.2 检查点

为在任务超时或机器故障后继续训练，以及事后分析训练过程或采样不同阶段的模型，需要保存检查点。检查点至少包含模型权重；有状态的优化器（如 AdamW）的状态；以及继续学习率调度所需的迭代次数。
nn.Module.state_dict() 和 torch.optim.Optimizer.state_dict() 返回状态字典，load_state_dict() 可恢复；torch.save(obj,dest) 保存对象到文件路径或类文件对象，torch.load(src) 再读回。

原文第 36 页：检查点与训练脚本

问题 checkpointing：实现模型检查点（1 分）实现：

- save_checkpoint(model, optimizer, iteration, out)：把模型、优化器及当前迭代次数的状态存到 out。可以将二者的 state_dict() 和迭代次数放入字典，调用 torch.save(obj, out)；只要能再加载，格式可自定。参数类型：model: torch.nn.Module，optimizer: torch.optim.Optimizer，iteration: int，out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]。
- load_checkpoint(src, model, optimizer)：从路径或类文件对象 src 读出检查点，恢复模型与优化器状态，并返回存下的迭代次数。参数类型：src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]，model: torch.nn.Module，optimizer: torch.optim.Optimizer。

完成 adapters.run_save_checkpoint 与 adapters.run_load_checkpoint，运行 uv run pytest -k test_checkpointing。

5.3 训练循环

编写主训练脚本，将此前的所有组件组合。之后会重复试验不同超参数，最好让脚本能轻松启动不同配置（例如通过命令行参数）。

问题 training_together：整合训练（4 分）编写使用用户提供输入训练模型的脚本。建议至少支持：配置模型和优化器各项超参数；通过 np.memmap 高效加载大型训练集与验证集；将检查点保存到用户指定路径；按固定间隔记录训练与验证表现（最后一项续见下一页）。

原文第 37 页：文本生成

training_together 还应定期记录训练和验证指标，可输出到控制台或 Weights & Biases（wandb.ai）等服务。

6 生成文本

语言模型接收长度为 sequence_length 的 token ID 序列（也可批量输入），输出一个 (sequence_length, vocab_size) 矩阵；每个位置给出对下一 token 的预测。以下把它变成新序列的采样流程。

Softmax：模型最终线性层的输出称为 logits，须通过前述 softmax（公式 10）转换为归一化概率。

解码：给模型一个前缀 token 序列（prompt），由最后一个位置的输出分布抽样下一个 token。公式 21 – 22：P(x_(t+1)=i | x_(1:t)) = exp(v_i)/Σ_j exp(v_j)，其中 v = TransformerLM(x_(1:t))_t ∈ ℝ^(vocab_size)。重复这个过程，每次把生成的 token 追加到输入，直到产生 <|endoftext|> 或达到用户指定的最大生成数。

解码技巧——温度：小模型生成质量可能较低。温度缩放把 softmax 改为（公式 23）softmax(v, τ)_i = exp(v_i/τ)/Σ_(j=1)^(vocab_size) exp(v_j/τ)。当 τ→0，最大 logit 占主导，分布趋近集中在该项的 one-hot 向量。

原文第 38 页：Top-p 采样与实验记录

核采样 / top-p：温度缩放后得到词表上的分布 q。取概率最大的若干 token，构成使其累计概率至少为 p 的最小索引集 V(p)；仅在此集合上重新归一化采样。公式 24：若 i∈V(p)，P(x_(t+1)=i|q)=q_i/Σ_(j∈V(p))q_j；否则为 0。可按 q 从大到小排序，选到累计概率达到 p 为止。

问题 decoding：解码（3 分）实现语言模型解码函数。建议支持：根据用户提供的 prompt 生成直至 <|endoftext|>；控制最大生成 token 数；按指定温度缩放下一 token 分布后采样；按用户指定阈值进行 top-p 核采样。

7 实验

现在把各部分整合，在预训练数据集上训练小型语言模型。理解架构组件最直接的方法是亲自修改并运行。为快速、稳定地比较实验，本作业使用约 1700 万参数的小模型和 TinyStories 数据集。要系统地消融组件、改变超参数，并记录实验与对应的学习曲线。为能提交损失曲线，应定期计算验证损失，记录训练步数与实际经过的时间；可使用 Weights & Biases 等记录工具。

问题 experiment_log：记录实验（3 分）在训练与评估代码中加入实验跟踪设施，以梯度更新步数和实际时间为横轴，记录实验与损失曲线；具体交付要求见下一页。

原文第 39 页：TinyStories 与基础超参数

experiment_log 的交付物：实验记录设施代码，以及本节后续题目的实验日志，写明你尝试的所有设置。

7.2 TinyStories

先使用简单、训练较快的 TinyStories 数据集，以便观察模型行为。获取方式见第 1 节。原文例子 tinystories_example 译如下：

> 从前，有个叫 Ben 的小男孩。Ben 喜欢探索周围的世界。他看见过许多奇妙的东西，比如商店里陈列的美丽花瓶。一天，Ben 在商店里走着，遇见了一个非常特别的花瓶。他看见花瓶时惊呆了！他说：“哇，这个花瓶真漂亮！我能买下它吗？”店主笑着说：“当然可以。你可以把它带回家，让所有朋友看看它有多漂亮！”于是 Ben 把花瓶带回家，心里十分自豪。他把朋友们叫来，向他们展示这个漂亮的花瓶。所有朋友都觉得它很美，也不敢相信 Ben 竟这么幸运。Ben 就这样在商店里找到了一个漂亮的花瓶！

7.2.1 调整超参数

- 词表大小：从 10,000 开始。常见词表大小为几万至几十万；尝试改变它，观察词表和模型行为。
- 上下文长度：256。TinyStories 这样的简单数据或许不需要长序列；后面的 OpenWebText 可尝试变化。观察单步运行时间和最终困惑度。
- d_model：512，略小于不少小型 Transformer 论文使用的 768，以提高速度。
- d_ff：1344，接近 $(8/3)d_{model}$ 且为 64 的倍数，有利于 GPU 性能。
- RoPE 参数：$\Theta=10000$。
- 层与头数：4 层、16 个头，共约 1700 万个非嵌入参数，属于较小的 Transformer。
- 处理的 token 总数：327,680,000；即 batch_size × 总步数 × context_length 约等于该值。

还须通过试验确定较好的学习率、预热长度、AdamW 的其他参数 $\beta_1, \beta_2, \varepsilon$ 与权重衰减率；可参考 Adam 论文中的常见设置。

7.2.2 整合训练

取得训练好的 BPE tokenizer，token 化训练数据，并用前述训练循环运行。若实现正确且高效，上述配置在一张 B200 GPU 上约需 20–30 分钟；后续提醒见下一页。

原文第 40 页：调试提示与学习率实验

若运行明显更慢，应检查数据加载、检查点保存、验证损失计算是否构成瓶颈，以及是否正确批量化。

7.2.3 调试模型架构的提示

熟悉 IDE（如 VS Code、Zed）的调试器通常比大量 `print` 更省时；文本编辑器可用 `ipdb`。常用检查方式：先尝试让模型在单个 mini-batch 上过拟合，正确实现通常能很快把训练损失降至接近零；在组件中设断点，检查中间张量形状；监视激活、权重与梯度的范数是否爆炸或消失。

问题 `learning_rate`：调整学习率（预计 2 B200 GPU 小时，3 分）

- (a) 对学习率作参数扫描，报告最终损失；若优化器发散则注明。交付：多个学习率对应的学习曲线和搜索策略说明；另须交付在 TinyStories 上每 token 验证损失不高于 1.45 的模型。
- 低资源提示：若在 CPU 或 Apple Silicon 上训练，将处理的 token 总数减至约 40,000,000，仍可生成大致流畅的文本；目标验证损失可放宽到 2.00。参考实测：M4 Max（36 GB 内存），$32 \times 5000 \times 256 = 40{,}960{,}000$ token，在 CPU 上约 1 小时 22 分钟、MPS 上约 36 分钟；第 5000 步验证损失为 1.80。若训练共 N 步，建议余弦学习率调度在第 N 步恰好到达最小值。在 MPS 上不要设置 `torch.set_float32_matmul_precision('high')` 来启用 TF32 内核；原文指出 PyTorch 2.9.0 的 MPS 后端有时会静默使用错误内核，造成训练不稳定。

原文第 41 页：学习率、批量大小与生成示例

低资源提示续：`torch.compile` 可加速训练。CPU 可用 `model = torch.compile(model)`；MPS 可用 `model = torch.compile(model, backend="aot_eager")` 在一定程度上优化反向传播。以原文编写时的 PyTorch 2.9.0 而言，MPS 不支持 Inductor 编译。

`learning_rate` 的 (b)：常有经验说最优学习率在“稳定边缘”。研究发散阈值与最佳学习率的关系。交付：一组逐渐增大学习率的学习曲线，其中至少一次发散，并分析与收敛速度的关系。

问题 `batch_size_experiment`：改变 batch 大小（预计 1 B200 GPU 小时，1 分）把 batch 大小从 1 一直变到 GPU 内存允许的上限，中间至少试几个值，包括 64、128 等常见值。必要时重新调整学习率。交付：各 batch 大小的学习曲线，以及几句话分析它对训练的影响。

接下来用解码器生成文本。原文示例 `ts_generate_example` 是一个至少达到预期质量的 TinyStories 模型输出，译如下（保留原输出的重复和不连贯之处）：

> 从前，有个漂亮的女孩叫 Lily。她喜欢吃口香糖，尤其喜欢那块大的黑色口香糖。一天，Lily 的妈妈请她帮忙做晚饭。Lily 非常兴奋！她喜欢帮妈妈。Lily 的妈妈做了一大锅汤当晚餐。Lily 高兴地说：“谢谢你，妈妈！我爱你。”她帮妈妈把汤倒进一个大碗。晚饭后，Lily 的妈妈又做了一些美味的汤。Lily 很喜欢！她说：“谢谢你，妈妈！这汤真好喝！”妈妈笑着说：“我很高兴你喜欢，Lily。”她们做完饭，又继续一起做饭。故事结束。

低资源提示：在 CPU 或 Apple Silicon 上生成文本，详情见下一页。

原文第 42 页：生成文本与归一化消融

如果用仅处理 4000 万 token 的低资源配置，生成文本仍应像英语，但流畅性可能不及上页。原文示例译如下；其中代词、语义和句子本身不连贯，应视为模型输出的特征：

> 从前，有个叫 Sue 的小女孩。Sue 有一颗她非常喜欢的牙齿。那是他最好的脑袋。一天，Sue 出门散步，遇见了一只瓢虫！他们成了好朋友，一起在小路上玩。 “嘿，Polly！我们出去吧！”Tim 说。Sue 抬头望天，发现很难找到一种闪耀着跳舞的办法。她笑了笑，同意帮助说话的东西！“ 当 Sue 看着天空移动时，那是什么。她

问题 `generate`：生成文本（1 分）使用自己的解码器和已训练检查点，报告模型生成内容。可能需要调整温度、top-p 等参数。交付：至少 256 个 token 的生成文本（若先遇到 `<|endoftext|>` 则到该处为止），并简短评价流畅性，指出至少两个影响质量的因素。

7.3 消融与架构修改

通过修改 Transformer 并观察行为，研究各组件的作用。

消融 1：层归一化。移除每个 Transformer block 中的 RMSNorm，观察先前最优学习率下会怎样；降低学习率能否恢复稳定？

问题 `layer_norm_ablation`：移除 RMSNorm 后训练（预计 0.5 B200 GPU 小时，1 分）交付：去掉 RMSNorm 后的学习曲线、该架构在最佳学习率下的曲线，以及几句话分析 RMSNorm 的影响。

Pre-norm block 的定义（公式 25–26）：`z = x + MultiHeadSelfAttention(RMSNorm(x))`，`y = z + FFN(RMSNorm(z))`。下一页将与 post-norm 比较。

原文第 43 页：归一化位置、位置编码与 SwiGLU 消融

原始 Transformer 使用 post-norm（公式 27 – 28）：$z = \mathrm{RMSNorm}(x + \mathrm{MultiHeadSelfAttention}(x))$，$y = \mathrm{RMSNorm}(z + \mathrm{FFN}(z))$。Pre-norm 是少数得到广泛共识的架构修改之一。现在改回 post-norm，观察训练行为。

问题 pre_norm_ablation：实现并训练 post-norm（预计 0.5 B200 GPU 小时，1 分）将 pre-norm Transformer 改为 post-norm 并训练。交付：post-norm 与 pre-norm 的学习曲线对比。层归一化及其放置位置都会明显影响模型行为。

消融 2：位置嵌入。比较使用 RoPE 的基准模型与完全不提供显式位置嵌入的 NoPE。理论上，带因果掩码的纯解码器 Transformer 即使没有显式位置编码，也可能推断相对或绝对位置信息；这里进行经验比较。

问题 no_pos_emb：实现 NoPE（预计 0.5 B200 GPU 小时，1 分）从使用 RoPE 的模型中彻底移除位置嵌入信息，观察结果。交付：RoPE 与 NoPE 性能的学习曲线对比。

消融 3：SwiGLU 与 SiLU。比较带门控的 SwiGLU 前馈网络，以及使用 SiLU 但不使用 GLU 的网络。后者（公式 29）为 $\mathrm{FFN_SiLU}(x) = W_2\,\mathrm{SiLU}(W_1 x)$。默认 SwiGLU 的内部维度约为 $(8/3)d_{model}$（并保证 $d_{ff} \bmod 64 = 0$，利用 GPU tensor core）；无门控的 SiLU 基线应设 $d_{ff} = 4d_{model}$，使两者参数量大致相当，因为后者只有两个权重矩阵，前者有三个。

问题 swiglu_ablation：SwiGLU 与 SiLU 对比（预计 0.5 B200 GPU 小时，1 分）具体交付要求见下一页。

原文第 44 页：OpenWebText 数据

swiglu_ablation 的交付物：在参数量近似相等的条件下比较 SwiGLU 与 SiLU 前馈网络的学习曲线，并用几句话讨论发现。

低资源提示：后续转向规模更大、噪声更多的网页数据集 OpenWebText，研究架构修改，也可选做课程排行榜。OpenWebText 上训练到流畅输出耗时较长，因此 GPU 资源有限的在线学习者可继续在 TinyStories 上测试修改，并用验证损失评估表现。

7.4 在 OpenWebText 上运行

这是一份从网页抓取内容制作、较典型的预训练数据集。课程提供了单个文本文件形式的 OpenWebText 小样本，获取方式见第 1 节。它比 TinyStories 更真实、更复杂、更多样。建议浏览训练文件，了解网页语料。原文例子 owt_example 译如下：

> 《Baseball Prospectus》的技术主管 Harry Pavlidis 雇用 Jonathan Judge 时冒了险。Pavlidis 知道，正如 Alan Schwarz 在 *The Numbers Game* 中写道，美国文化中没有哪个领域像棒球运动员的表现那样被如此精确地计数、如此热切地量化。点几下鼠标，你就能查到 Noah Syndergaard 的快球飞向本垒板时每分钟旋转超过 2100 次，Nelson Cruz 在 2016 年符合统计资格的击球手中拥有最高的平均击球初速，以及许多仿佛来自电子游戏或科幻小说的小知识。不断上涨的数据海洋让棒球文化中一个愈发重要的角色获得了能力：业余数据分析爱好者。 这种能力也带来了更多审视：既针对测量结果，也针对其背后的人和出版物。在《Baseball Prospectus》工作，Pavlidis 很清楚定量分析不够完美时会遭遇的反弹。他也知道网站的接球指标需要重做，而完成这项工作需要一个学识丰富、能解决复杂统计建模问题的人。 “他把我们吓坏了。”——Harry Pavlidis。Pavlidis 根据 Judge 的文章，以及两人在网站主办的球场活动上的交流，直觉认为 Judge “懂行”。〔原文在此省略后文〕

原文第 45 页：OpenWebText 实验与排行榜规则

这一实验可能需要重新调整学习率、batch 大小等超参数。

问题 main_experiment：OpenWebText 实验（预计 2 B200 GPU 小时，2 分）采用与 TinyStories 相同的模型架构及训练迭代总数，在 OpenWebText 上训练语言模型。效果如何？交付：OpenWebText 学习曲线；说明与 TinyStories 的损失差异，以及该如何解释。另按 TinyStories 相同格式交付模型生成文本，分析其流畅性；为什么使用相同模型和算力预算，输出质量仍较差？

7.5 自行修改与排行榜

最后尝试改进 Transformer 架构，并与其他同学比较超参数与架构效果。

排行榜规则：运行时间最多为一张 B200 上的 45 分钟；若用 SLURM 或 Modal，建议在提交脚本中确保不超时。数据只能使用课程提供的 OpenWebText 训练集。除此之外可以自由尝试。

可参考 Llama 3、Qwen 2.5 等开源模型家族，以及 NanoGPT speedrun 仓库 github.com/KellerJordan/modded-nanogpt 中小规模预训练的各种修改。一个常见修改是让输入嵌入与输出嵌入共用权重；若尝试权重绑定，可能需要降低 embedding/LM head 初始化的标准差。建议先在 OpenWebText 小子集或 TinyStories 上测试，再跑完整的 45 分钟。这里表现好的改动未必能推广到更大规模的预训练，课程的 scaling laws 单元会继续讨论。

问题 leaderboard：排行榜（预计 10 B200 GPU 小时，6 分）具体目标和交付要求见下一页。

原文第 46 页：排行榜交付与参考文献（上）

按上述规则训练，目标是在 0.75 B200-hours 内尽量降低语言模型验证损失。交付：最终记录的验证损失；一条以实际经过时间为横轴、清楚显示总时间少于 45 分钟的学习曲线；以及所做修改的说明。预期成绩至少优于验证损失 5.0 的朴素基线。排行榜提交地址：github.com/stanford-cs336/assignment1-basics-leaderboard。

参考文献

文献作者、标题与出处保留英文，便于检索。

1. R. Eldan and Y. Li, “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?.” 2023.
2. A. Gokaslan, V. Cohen, E. Pavlick, and S. Tellex, “OpenWebText corpus.” 2019.
3. R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in Proc. of ACL, 2016.
4. C. Wang, K. Cho, and J. Gu, “Neural Machine Translation with Byte-Level Subwords.” 2019.
5. P. Gage, “A new algorithm for data compression,” C Users Journal, vol. 12, no. 2, pp. 23–38, Feb. 1994.
6. A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners.” 2019.
7. A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training.” 2018.
8. A. Vaswani et al., “Attention is All you Need,” in Proc. of NeurIPS, 2017.
9. T. Q. Nguyen and J. Salazar, “Transformers without Tears: Improving the Normalization of Self-Attention,” in Proc. of IWSWLT, 2019.
10. R. Xiong et al., “On Layer Normalization in the Transformer Architecture,” in Proc. of ICML, 2020.
11. J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization.” 2016.
12. H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models.” 2023.
13. B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” in Proc. of NeurIPS, 2019.
14. A. Grattafiori et al., “The Llama 3 Herd of Models.” https://arxiv.org/abs/2407.21783.
15. A. Yang et al., “Qwen2.5 Technical Report,” arXiv:2412.15115, 2024.
16. A. Chowdhery et al., “PaLM: Scaling Language Modeling with Pathways.” 2022.
17. D. Hendrycks and K. Gimpel, “Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units.” 2016.
18. S. Elfwing, E. Uchibe, and K. Doya, “Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning.” https://arxiv.org/abs/1702.03118.

19. Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, “Language Modeling with Gated Convolutional Networks.” https://arxiv.org/abs/1612.08083.

20. N. Shazeer, “GLU Variants Improve Transformer.” 2020.

原文第 47 页：参考文献（下）

21. J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding.” 2021.

22. D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in Proc. of ICLR, 2015.

23. I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in Proc. of ICLR, 2019.

24. T. B. Brown et al., “Language Models are Few-Shot Learners,” in Proc. of NeurIPS, 2020.

25. J. Kaplan et al., “Scaling Laws for Neural Language Models.” 2020.

26. J. Hoffmann et al., “Training Compute-Optimal Large Language Models.” 2022.

27. A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The Curious Case of Neural Text Degeneration,” in Proc. of ICLR, 2020.

28. Y.-H. H. Tsai, S. Bai, M. Yamada, L.-P. Morency, and R. Salakhutdinov, “Transformer Dissection: An Unified Understanding for Transformer`s Attention via the Lens of Kernel,” in EMNLP-IJCNLP, 2019, pp. 4344 – 4353. doi: 10.18653/v1/D19-1443.

29. A. Kazemnejad, I. Padhi, K. Natesan, P. Das, and S. Reddy, “The Impact of Positional Encoding on Length Generalization in Transformers,” in NeurIPS, 2023. https://openreview.net/forum?id=Drrl2gcjzl.